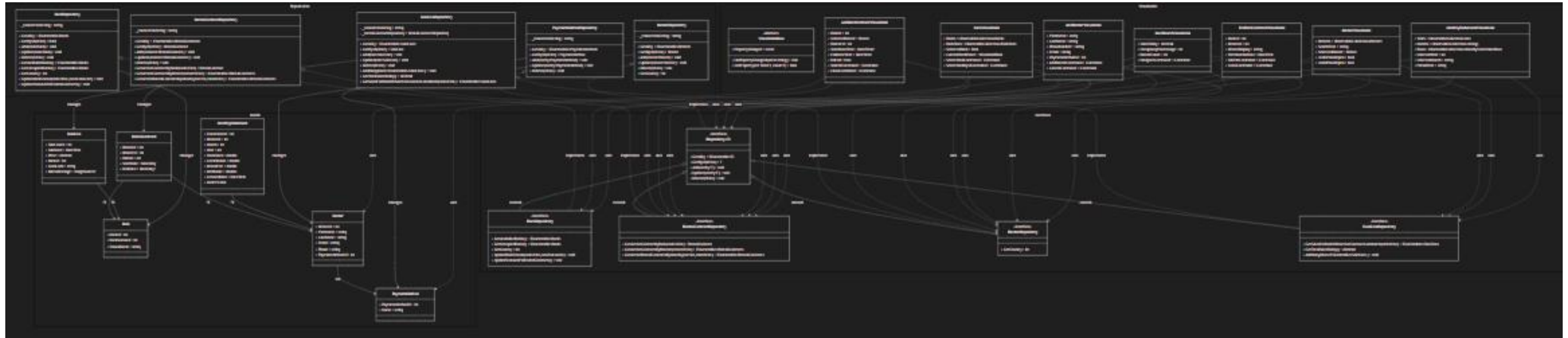


DCD for hele projektet	2
UC1: “Opret ny lejer”	3
SD	3
DCD	3
UC2: “Find lejer”	4
SD:	4
DCD:	4
UC3: “Se reolkartotek”	5
DCD	5
UC4: “Opret lejekontrakt”	6
DCD	6
UC5: “Opsig lejekontrakt”	7
DCD	7
UC6: “Opret labels”	8
DCD	8
UC7: “Registrer salg af vare”	9
DCD	9
UC8: “Månedlig opgørelse”	10
DCD	10
Relationsskema	11
UML-databasemodel	12
Database-script	13
ER-diagram	20
Mermaid-kode	21

DCD for hele projektet

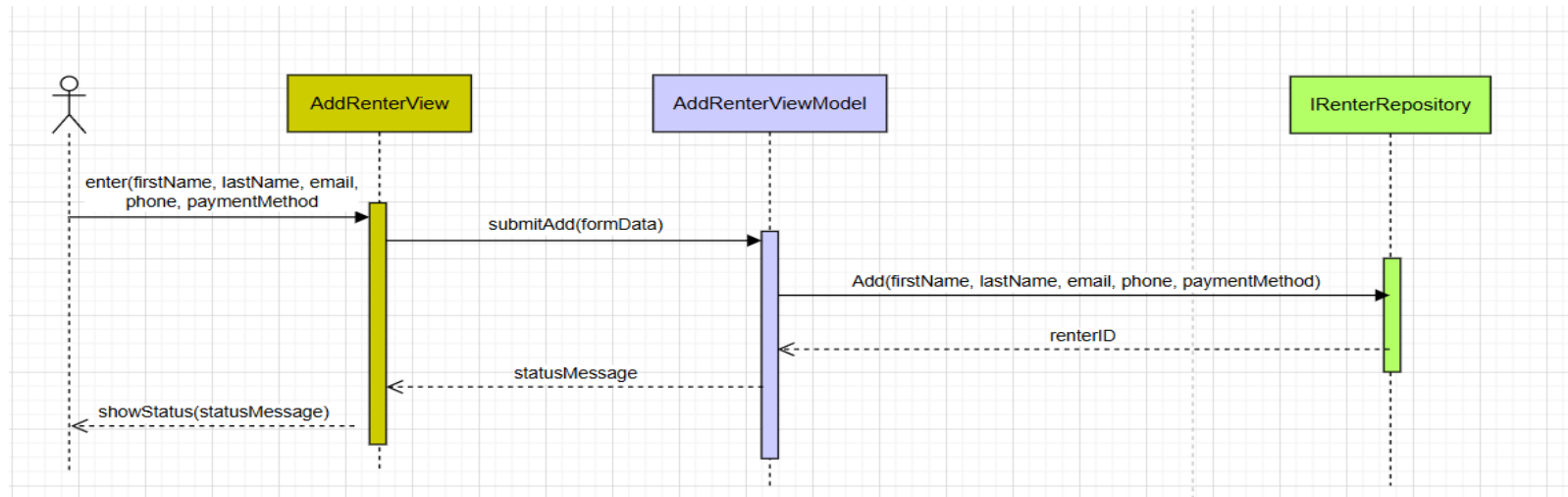


DCD for hele projektet ligger i vores VS-projekt i mappen “documentation”, og mermaid koden kan også findes sidst i dette dokument (side 18).

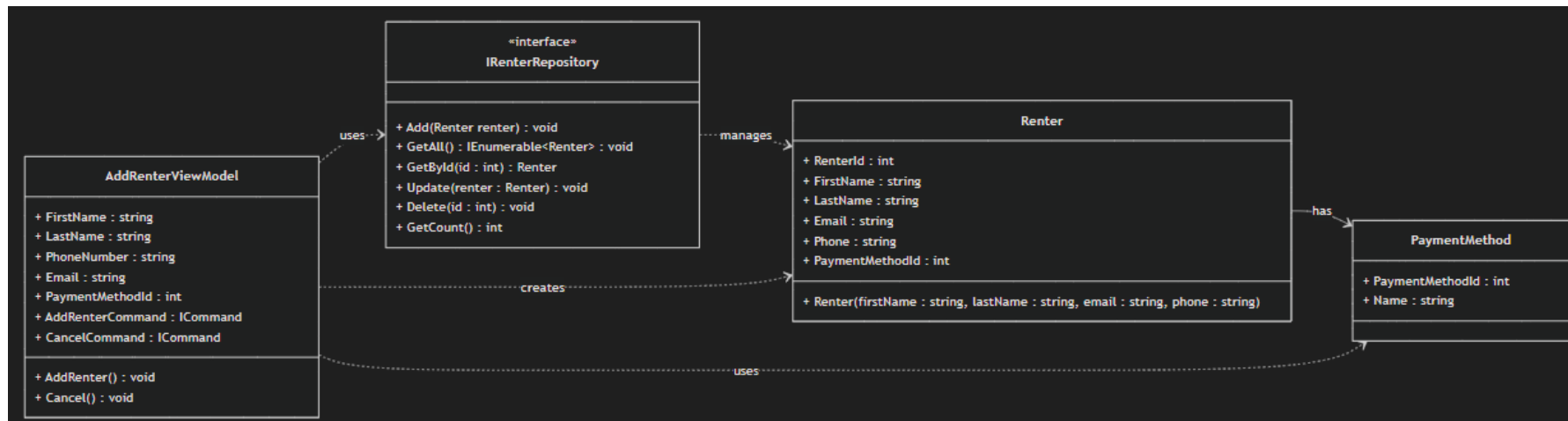
For overskuelighedens skyld, har vi valgt at dele DCD op for hver use case.

UC1: “Opret ny lejer”

SD

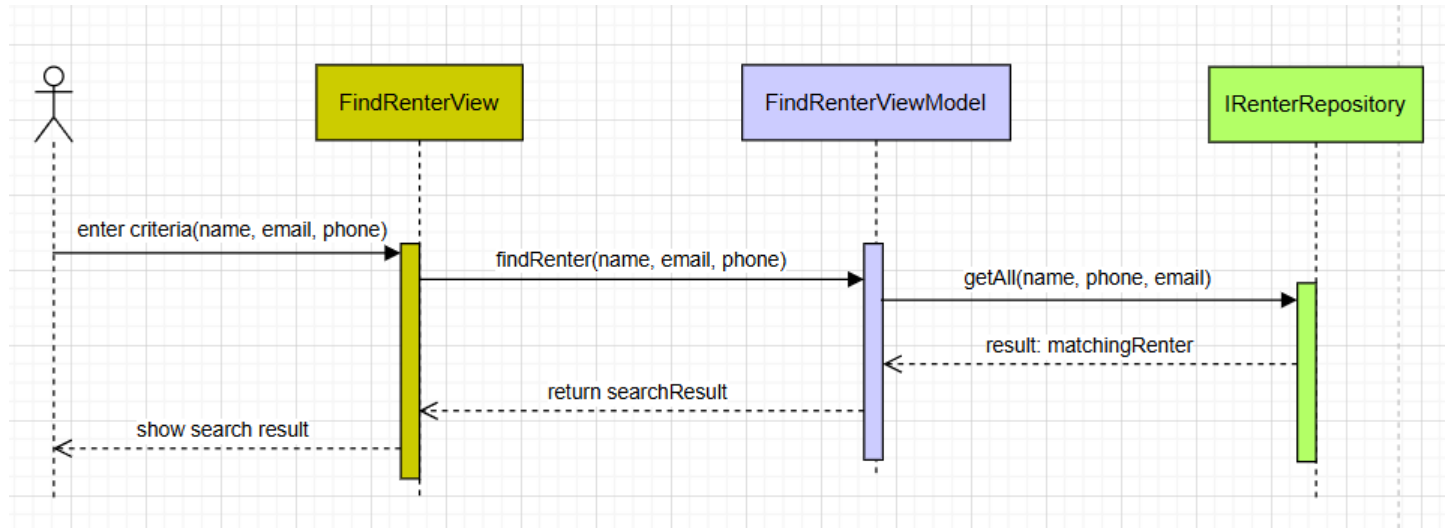


DCD

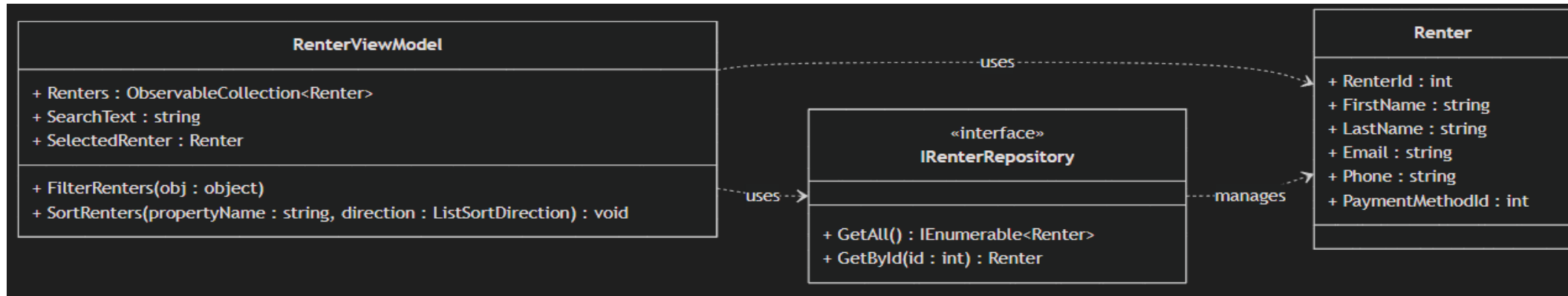


UC2: “Find lejer”

SD:

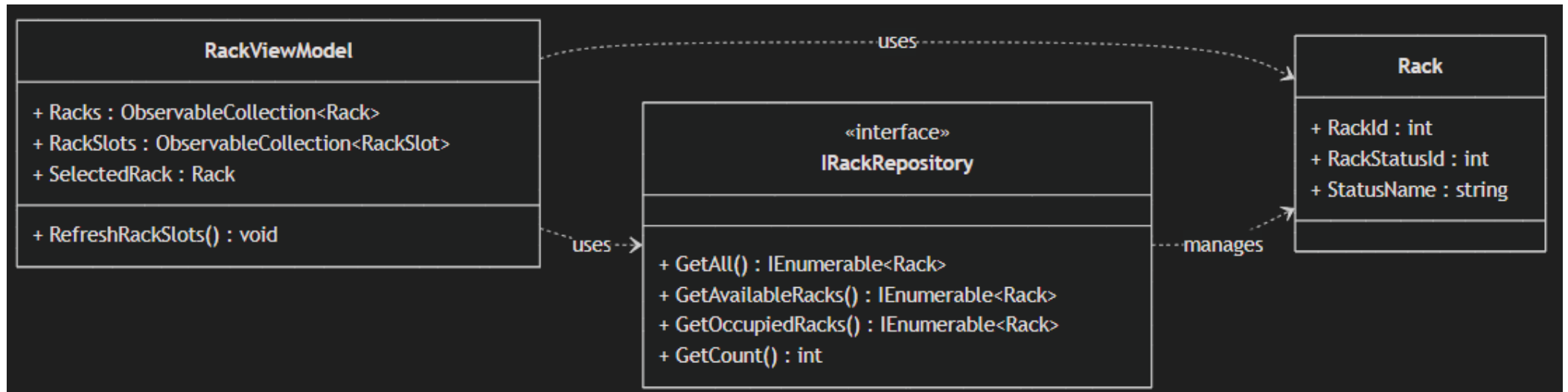


DCD:



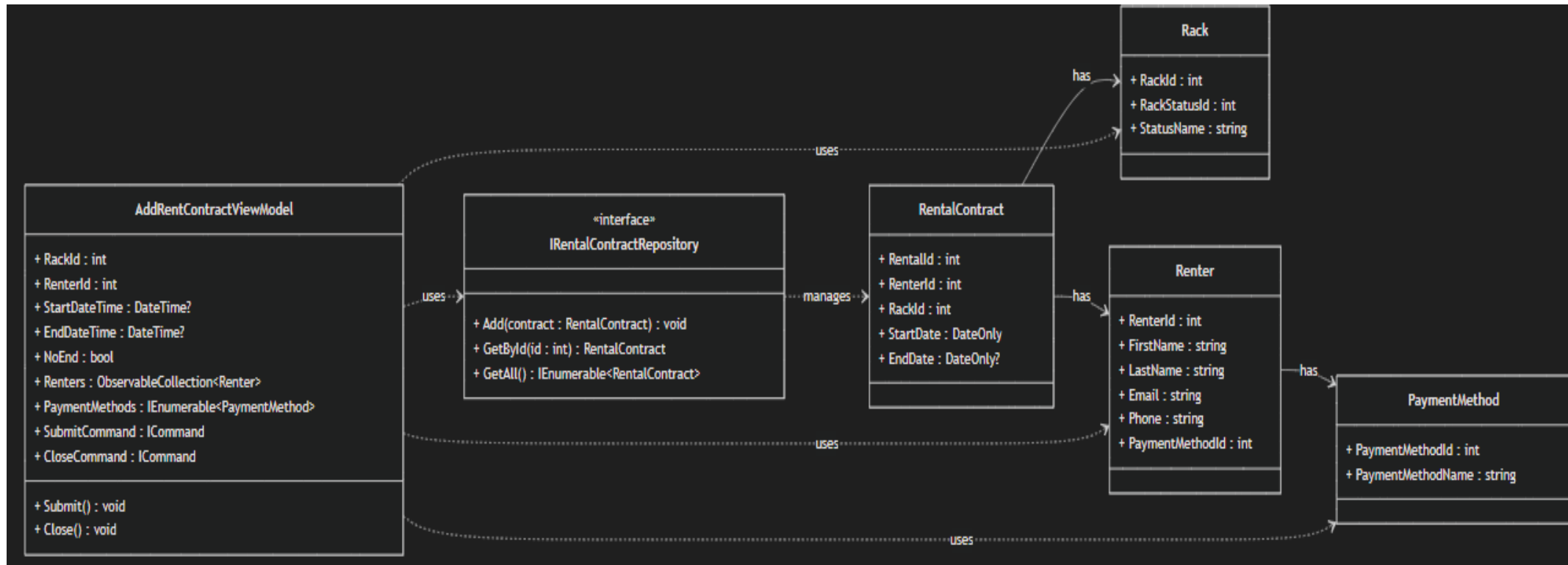
UC3: “Se reolkartotek”

DCD



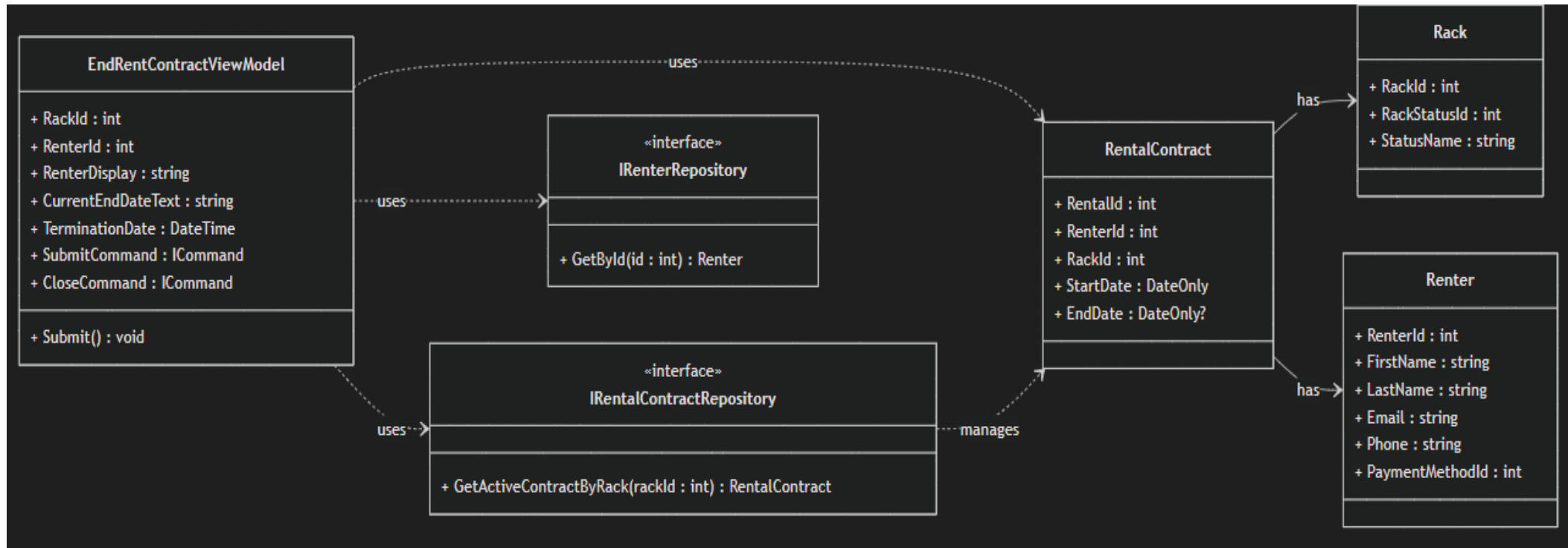
UC4: “Opret lejekontrakt”

DCD



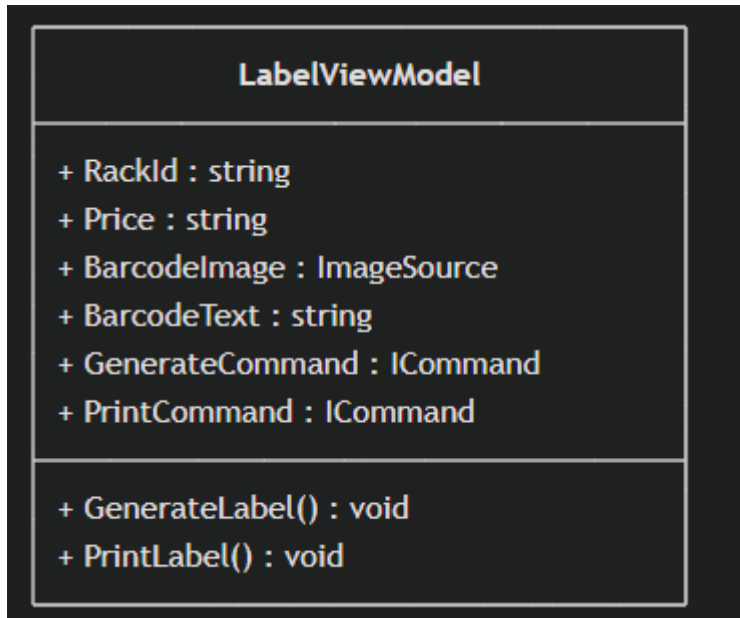
UC5: “Opsig lejekontrakt”

DCD



UC6: “Opret labels”

DCD



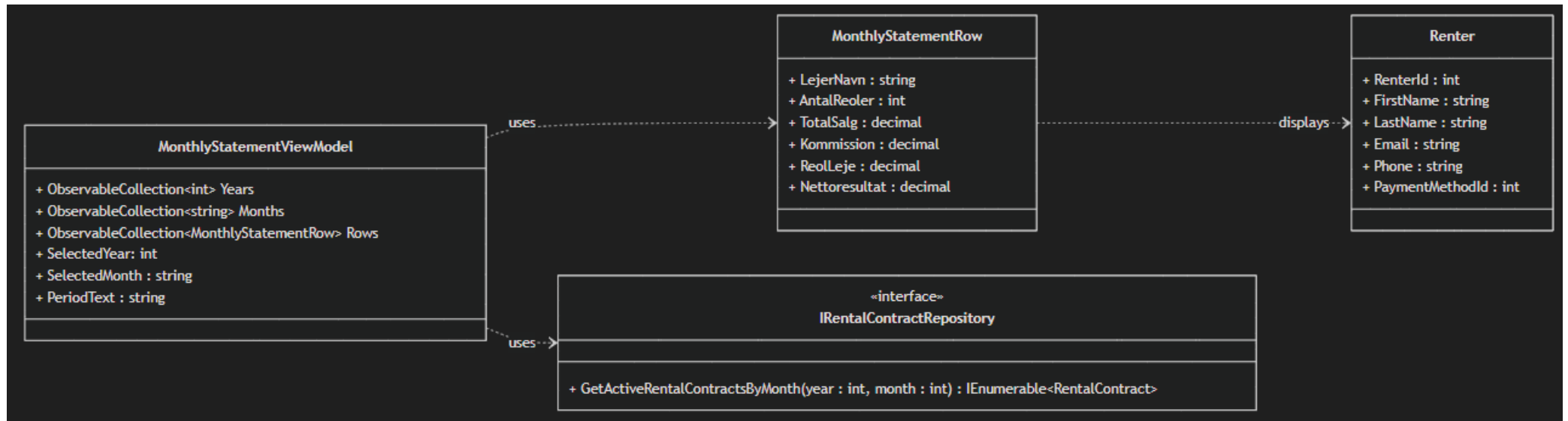
UC7: “Registrer salg af vare”

DCD



UC8: “Månedlig opgørelse”

DCD



Relationsskema

PAYMENTMETHOD(PaymentMethodId, PaymentMethodName)

RENTER (RenterId, FirstName, LastName, Phone, Email, *PaymentMethodId*)

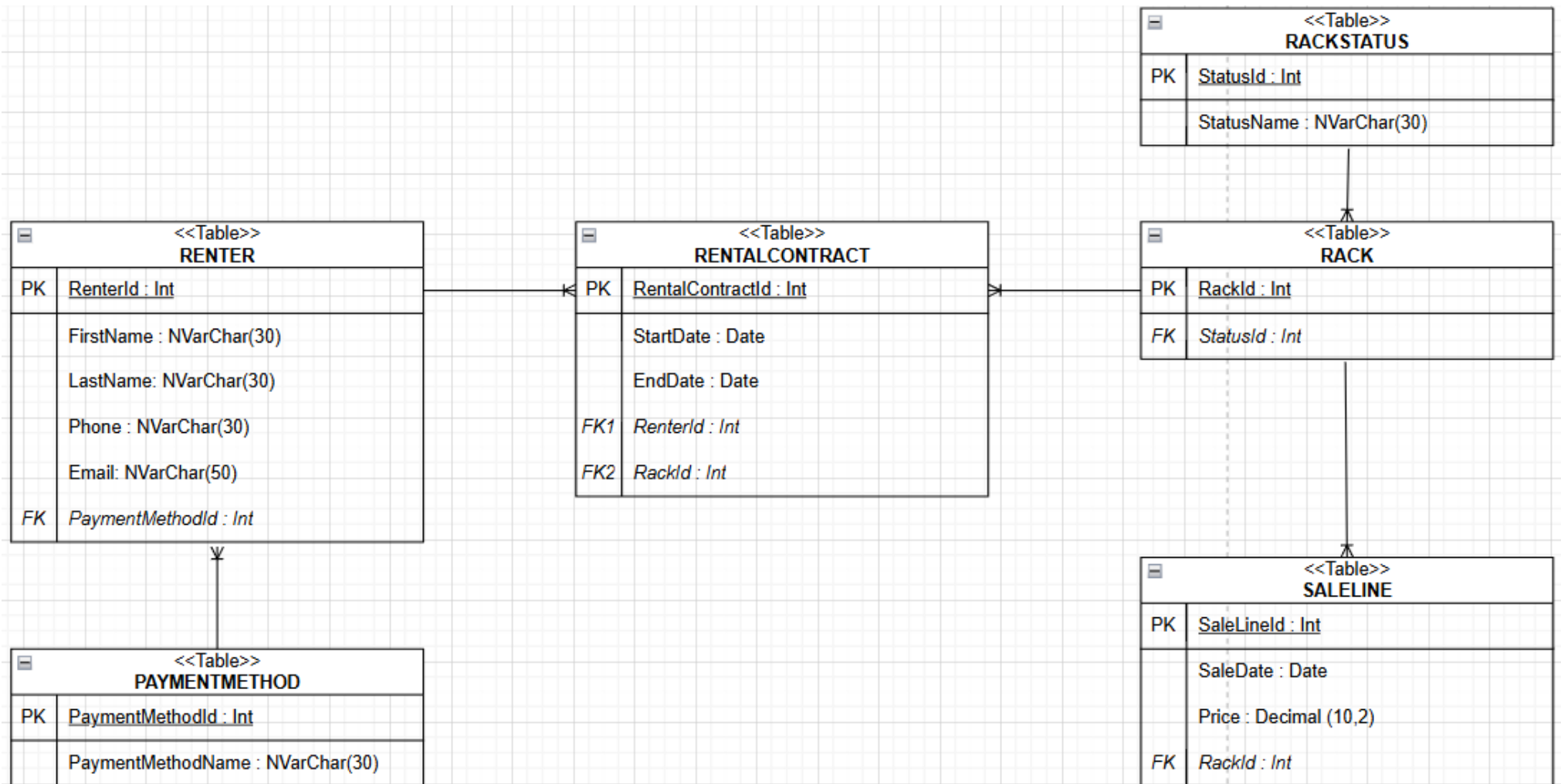
RACKSTATUS(StatusID, StatusName)

RACK(RackId, *StatusId*)

RENTALCONTRACT(RentalContractId, StartDate, EndDate, *RenterId*, *RackId*)

SALELINE(SaleLineId, SaleDate, Price, *RackId*)

UML-databasemodel



Database-script

(Findes også som tekst-fil i mappen “Documentation” i Visual Studio projektet)

-- Opret database

```
CREATE DATABASE ReolmarkedDB;  
GO
```

```
USE ReolmarkedDB;  
GO
```

-- Opret PaymentMethod

```
CREATE TABLE PaymentMethod (  
    PaymentMethodId INT IDENTITY(1,1) PRIMARY KEY,  
    PaymentMethodName NVARCHAR(30) NOT NULL );  
GO
```

-- Opret RackStatus

```
CREATE TABLE RackStatus (  
    RackStatusId INT IDENTITY(1,1) PRIMARY KEY,  
    StatusName NVARCHAR(30) NOT NULL );  
GO
```

-- Opret Rack

```
CREATE TABLE Rack (  
    RackId INT PRIMARY KEY,  
    RackStatusId INT NOT NULL,  
    FOREIGN KEY (RackStatusId) REFERENCES RackStatus(RackStatusId) );  
GO
```

-- Opret Renter

```
CREATE TABLE Renter (  
    RenterId INT IDENTITY(1,1) PRIMARY KEY,  
    FirstName NVARCHAR(30) NOT NULL,  
    LastName NVARCHAR(30) NOT NULL,
```

```
Phone NVARCHAR(20) UNIQUE,  
Email NVARCHAR(50) UNIQUE,  
PaymentMethodId INT NOT NULL,  
FOREIGN KEY (PaymentMethodId) REFERENCES PaymentMethod(PaymentMethodId) );
```

GO

-- Opret RentalContract

```
CREATE TABLE RentalContract (  
    RentalContractId INT IDENTITY(1,1) PRIMARY KEY,  
    StartDate DATE NOT NULL,  
    EndDate DATE NULL,  
    RenterId INT NOT NULL,  
    RackId INT NOT NULL,  
    FOREIGN KEY (RenterId) REFERENCES Renter(RenterId),  
    FOREIGN KEY (RackId) REFERENCES Rack(RackId),  
    CHECK (EndDate > StartDate) );
```

GO

-- Opret SaleLine

```
CREATE TABLE SaleLine (  
    SaleLineId INT IDENTITY(1,1) PRIMARY KEY,  
    SaleDate DATE NOT NULL,  
    Price DECIMAL(8,2) NOT NULL CHECK (Price >= 0),  
    RackId INT NOT NULL,  
    FOREIGN KEY (RackId) REFERENCES Rack(RackId) );
```

GO

-- Indsæt betalingsmetoder

```
INSERT INTO PaymentMethod (PaymentMethodName) VALUES  
('MobilePay'), ('Bankoverførsel'), ('Kort');
```

GO

-- Indsæt rackstatus

```
INSERT INTO RackStatus (StatusName) VALUES
```

```
('Ledig'), ('Optaget');  
GO
```

```
-- Indsæt renters
```

```
INSERT INTO Renter (FirstName, LastName, Phone, Email, PaymentMethodId) VALUES  
( 'Anna', 'Jensen', '11111111', 'anna.jensen@example.com', 1),  
( 'Bo', 'Hansen', '22222222', 'bo.hansen@example.com', 2),  
( 'Clara', 'Nielsen', '33333333', 'clara.nielsen@example.com', 3),  
( 'David', 'Larsen', '44444444', 'david.larsen@example.com', 1),  
( 'Eva', 'Madsen', '55555555', 'eva.madsen@example.com', 2),  
( 'Frederik', 'Olsen', '66666666', 'frederik.olsen@example.com', 3),  
( 'Grethe', 'Thomsen', '77777777', 'grethe.thomsen@example.com', 1),  
( 'Henrik', 'Christensen', '88888888', 'henrik.christensen@example.com', 2),  
( 'Ida', 'Andersen', '99999999', 'ida.andersen@example.com', 3),  
( 'Jakob', 'Mortensen', '10101010', 'jakob.mortensen@example.com', 1),  
( 'Katrine', 'Poulsen', '12121212', 'katrine.poulsen@example.com', 2),  
( 'Lars', 'Pedersen', '13131313', 'lars.pedersen@example.com', 3),  
( 'Maria', 'Knudsen', '14141414', 'maria.knudsen@example.com', 1),  
( 'Niels', 'Berg', '15151515', 'niels.berg@example.com', 2),  
( 'Olivia', 'Frandsen', '16161616', 'olivia.frandsen@example.com', 3),  
( 'Peter', 'Vestergaard', '17171717', 'peter.vestergaard@example.com', 1),  
( 'Rikke', 'Holm', '18181818', 'rikke.holm@example.com', 2),  
( 'Søren', 'Winther', '19191919', 'soeren.winther@example.com', 3),  
( 'Tina', 'Brandt', '20202020', 'tina.brandt@example.com', 1),  
( 'Ulrik', 'Krogh', '21212121', 'ulrik.krogh@example.com', 2),  
( 'Vibeke', 'Lund', '22223333', 'vibeke.lund@example.com', 3),  
( 'William', 'Nørgaard', '23232323', 'william.norgaard@example.com', 1),  
( 'Xenia', 'Bach', '24242424', 'xenia.bach@example.com', 2),  
( 'Yvonne', 'Skov', '25252525', 'yvonne.skov@example.com', 3),  
( 'Zacharias', 'Møller', '26262626', 'zacharias.moeller@example.com', 1);  
GO
```

```
-- Opret 80 reoler (hvis de ikke allerede findes)
```

```
DECLARE @rackId INT = 1;  
WHILE @rackId <= 80  
BEGIN
```

```
IF NOT EXISTS (SELECT 1 FROM Rack WHERE RackId = @rackId)
BEGIN
    INSERT INTO Rack (RackId, RackStatusId) VALUES (@rackId, 1);
END
SET @rackId += 1;
END;
GO
```

-- Randomisér 43 reoler som optaget ;

```
WITH Randomized AS (
    SELECT RackId, ROW_NUMBER() OVER (ORDER BY NEWID()) AS rn
    FROM Rack
)
UPDATE R
SET RackStatusId = CASE WHEN rn <= 43 THEN 2 ELSE 1 END
FROM Rack R
JOIN Randomized RR ON R.RackId = RR.RackId;
GO
```

-- Indsæt 43 kontrakter på de optagede reoler

```
DECLARE @Occupied TABLE (RowNum INT, RackId INT);

INSERT INTO @Occupied

SELECT ROW_NUMBER() OVER (ORDER BY NEWID()), RackId

FROM Rack WHERE RackStatusId = 2;

DECLARE @i INT = 1;
```



```
WHILE @i <= 43

BEGIN

    DECLARE @rack INT = (SELECT RackId FROM @Occupied WHERE RowNum = @i);

    DECLARE @renter INT = ((@i - 1) % (SELECT COUNT(*) FROM Renter)) + 1;

    DECLARE @startDate DATE = DATEADD(MONTH, ((@i - 1) % 5), '2025-05-01');

    INSERT INTO RentalContract (StartDate, EndDate, RenterId, RackId)

    VALUES (@startDate, NULL, @renter, @rack);

    SET @i += 1;

END;

GO
```

-- Indsæt 50 salgsdata for september fordelt på optagede reoler

```
SET NOCOUNT ON;
DECLARE @j INT = 1;
WHILE @j <= 50
BEGIN
    DECLARE @rack INT = (
        SELECT RackId FROM (
            SELECT RackId, ROW_NUMBER() OVER (ORDER BY NEWID()) AS rn
            FROM Rack WHERE RackStatusId = 2
        ) t WHERE rn = ((@j - 1) % 43) + 1
    );

    DECLARE @day INT = ((@j - 1) % 30) + 1;
    DECLARE @price DECIMAL(8,2) = ROUND(50 + RAND() * 150, 2);

    INSERT INTO SaleLine (SaleDate, Price, RackId)
    VALUES (DATEFROMPARTS(2025, 9, @day), @price, @rack);

    SET @j += 1;
END;
```

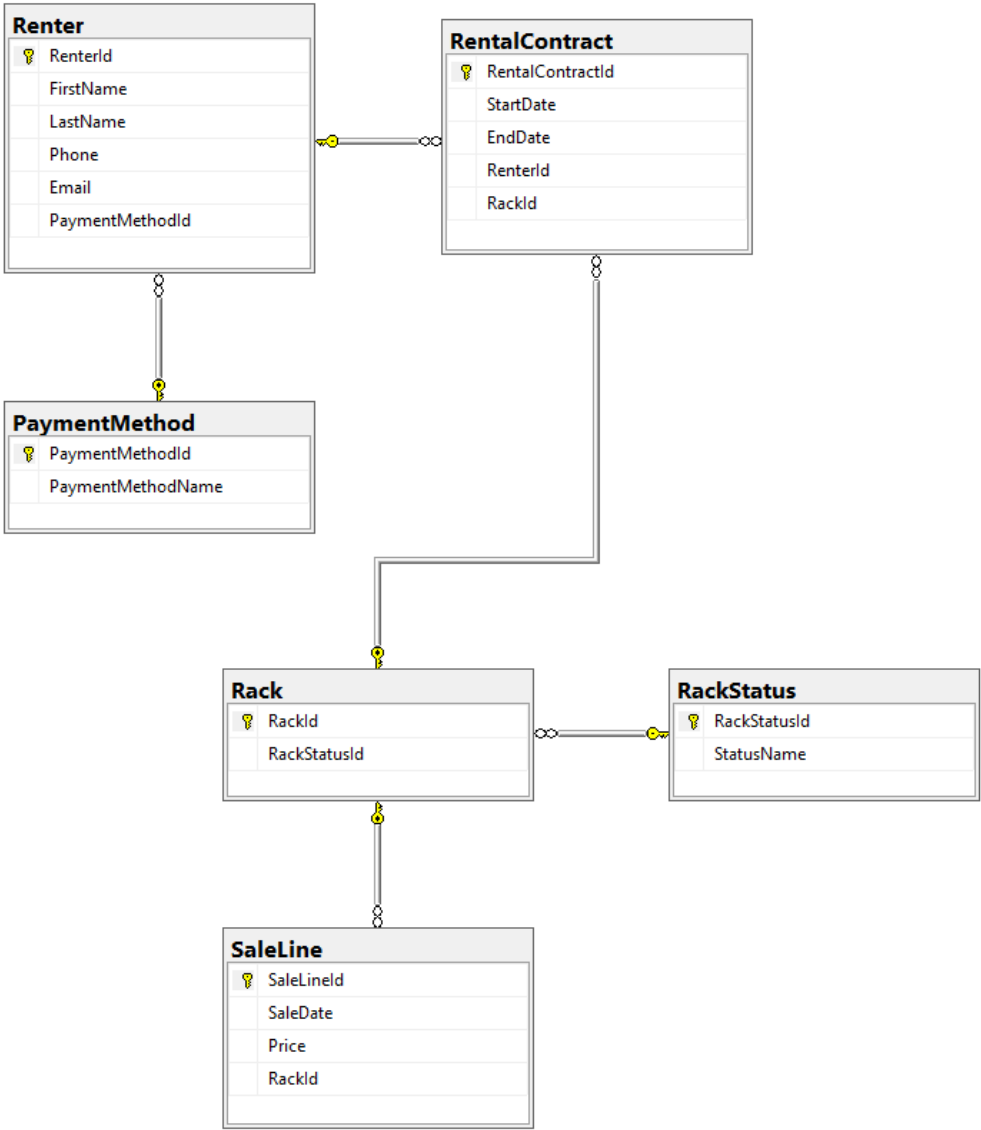
-- Validering

```
SELECT RackStatusId, COUNT(*) AS Antal FROM Rack GROUP BY RackStatusId;
SELECT * FROM RentalContract;
SELECT COUNT() AS Salg FROM SaleLine;
```

-- Se alle reoler med statusnavn

```
SELECT
    R.RackId,
    R.RackStatusId,
    RS.StatusName
FROM Rack R
INNER JOIN RackStatus RS ON R.RackStatusId = RS.RackStatusId
ORDER BY R.RackId;
GO
```


ER-diagram



Mermaid-kode

classDiagram

direction LR

```
class MonthlyStatement {  
    + StatementId : int  
    + RenterId : int  
    + Month : int  
    + Year : int  
    + TotalSales : double  
    + Commission : double  
    + RentalFee : double  
    + NetResult : double  
    + CreatedDate : DateTime  
    + enum Status  
}
```

```
class PaymentMethod {  
    + PaymentMethodId : int  
    + Name : string  
}
```

```
class Rack {  
    + RackId : int
```

```
    + RackStatusId : int  
    + StatusName : string  
}
```

```
class RentalContract {  
    + RentalId : int  
    + RenterId : int  
    + RackId : int  
    + StartDate : DateOnly  
    + EndDate : DateOnly?  
}
```

```
class Renter {  
    + RenterId : int  
    + FirstName : string  
    + LastName : string  
    + Email : string  
    + Phone : string  
    + PaymentMethodId : int  
}
```

```
class SaleLine {  
    + SaleLineId : int  
    + SaleDate : DateTime
```

```
+ Price : decimal
+ RackId : int
+ ScanCode : string
+ BarcodeImage : ImageSource?
}
```

```
class IRepository<T>{
    <<interface>>
    + GetAll() IEnumerable<T>
    + GetById(id:int) T
    + Add(entity:T) void
    + Update(entity:T) void
    + Delete(id:int) void
}
```

```
class IRackRepository {
    <<interface>>
    + GetAvailableRacks() IEnumerable<Rack>
    + GetOccupiedRacks() IEnumerable<Rack>
    + GetCount() int
    + UpdateRackStatus(rackId:int,newStatus:int) void
    + UpdateStatusesForEndedContracts() void
}
```

```
class IRentalContractRepository {  
    <<interface>>  
    + GetActiveContractByRack(rackId:int) RentalContract  
    + GetActiveContractsByRenter(renterId:int) IEnumerable~RentalContract~  
    + GetActiveRentalContractsByMonth(year:int,month:int) IEnumerable~RentalContract~  
}
```

```
class IRenterRepository {  
    <<interface>>  
    + GetCount() int  
}
```

```
class ISaleLineRepository {  
    <<interface>>  
    + GetSalesForRackWithActiveContractLastMonth(rackId:int) IEnumerable~SaleLine~  
    + GetTotalSalesToday() decimal  
    + AddMany(lines:IEnumerable~SaleLine~) void  
}
```

```
class PaymentMethodRepository {  
    - _connectionString : string  
    + GetAll() IEnumerable~PaymentMethod~
```



```
+ GetById(id:int) PaymentMethod
+ Add(entity:PaymentMethod) void
+ Update(entity:PaymentMethod) void
+ Delete(id:int) void
}
```

```
class RackRepository {
    - _connectionString : string
    + GetAll() IEnumerable~Rack~
    + GetById(id:int) Rack
    + Add(rack:Rack) void
    + Update(rack:Rack) void
    + Delete(id:int) void
    + GetAvailableRacks() IEnumerable~Rack~
    + GetOccupiedRacks() IEnumerable~Rack~
    + GetCount() int
    + UpdateRackStatus(rackId:int,newStatus:int) void
    + UpdateStatusesForEndedContracts() void
}
```

```
class RentalContractRepository {
    - _connectionString : string
    + GetAll() IEnumerable~RentalContract~
}
```

```
+ GetById(id:int) RentalContract
+ Add(contract:RentalContract) void
+ Update(contract:RentalContract) void
+ Delete(id:int) void
+ GetActiveContractByRack(rackId:int) RentalContract
+ GetActiveContractsByRenter(renterId:int) IEnumerable~RentalContract~
+ GetActiveRentalContractsByMonth(year:int,month:int) IEnumerable~RentalContract~
}
```

```
class RenterRepository {
    - _connectionString : string
    + GetAll() IEnumerable~Renter~
    + GetById(id:int) Renter
    + Add(renter:Renter) void
    + Update(renter:Renter) void
    + Delete(id:int) void
    + GetCount() int
}
```

```
class SaleLineRepository {
    - _connectionString : string
    - _rentalContractRepository : RentalContractRepository
    + GetAll() IEnumerable~SaleLine~
}
```

```
+ GetById(id:int) SaleLine
+ Add(line:SaleLine) void
+ Update(line:SaleLine) void
+ Delete(id:int) void
+ AddMany(lines:IEnumerable~SaleLine~) void
+ GetTotalSalesToday() decimal
+ GetSalesForRackWithActiveContractLastMonth(rackId:int) IEnumerable~SaleLine~
}
```

```
class ViewModelBase {
    <<abstract>>
    + PropertyChanged : event
    + OnPropertyChanged(name:string) void
    + SetProperty<T>(ref field:T, value:T) bool
}
```

```
class AddRenterViewModel {
    + FirstName : string
    + LastName : string
    + PhoneNumber : string
    + Email : string
    + PaymentMethodId : int
    + AddRenterCommand : ICommand
}
```

```
+ CancelCommand : ICommand  
}
```

```
class AddRentContractViewModel {  
    + RackId : int  
    + SelectedRenter : Renter  
    + RenterId : int  
    + StartDateTime : DateTime?  
    + EndDateTime : DateTime?  
    + NoEnd : bool  
    + SubmitCommand : ICommand  
    + CloseCommand : ICommand  
}
```

```
class DashBoardViewModel {  
    + SalesToday : decimal  
    + OccupancyPercentage : int  
    + RenterCount : int  
    + NavigateCommand : ICommand  
}
```

```
class EndRentContractViewModel {  
    + RackId : int
```

```
+ RenterId : int
+ RenterDisplay : string
+ TerminationDate : DateTime
+ SubmitCommand : ICommand
+ CloseCommand : ICommand
}
```

```
class RackViewModel {
    + Racks : ObservableCollection~Rack~
    + RackSlots : ObservableCollection~RackSlot~
    + SelectedRack : Rack
    + CurrentRackPanel : ViewModelBase
    + SelectRackCommand : ICommand
    + SelectRackByIdCommand : ICommand
}
```

```
class RenterViewModel {
    + Renters : ObservableCollection~Renter~
    + SearchText : string
    + SelectedRenter : Renter
    + IsInfoPanelOpen : bool
    + IsAddPanelOpen : bool
}
```

```
class MonthlyStatementViewModel {  
    + Years : ObservableCollection~int~  
    + Months : ObservableCollection~string~  
    + Rows : ObservableCollection~MonthlyStatementRow~  
    + SelectedYear : int  
    + SelectedMonth : string  
    + PeriodText : string  
}  
}
```

Renter --> PaymentMethod : has

RentalContract --> Renter : FK

RentalContract --> Rack : FK

SaleLine --> Rack : FK

MonthlyStatement --> Renter : FK

IRepository~T~ <|-- IRackRepository : extends

IRepository~T~ <|-- IRentalContractRepository : extends

IRepository~T~ <|-- IRenterRepository : extends

IRepository~T~ <|-- ISaleLineRepository : extends

PaymentMethodRepository <|.. IRepository~PaymentMethod~ : implements

RackRepository <|.. IRackRepository : implements

RentalContractRepository <|.. IRentalContractRepository : implements

RenterRepository <|.. IRenterRepository : implements

SaleLineRepository <|.. ISaleLineRepository : implements

PaymentMethodRepository --> PaymentMethod : manages

RackRepository --> Rack : manages

RentalContractRepository --> RentalContract : manages

RenterRepository --> Renter : manages

SaleLineRepository --> SaleLine : manages

AddRenterViewModel ..> IRenterRepository : uses

AddRenterViewModel ..> IRepository~PaymentMethod~ : uses

AddRentContractViewModel ..> Renter : uses

AddRentContractViewModel ..> PaymentMethod : uses

AddRentContractViewModel ..> IRentalContractRepository : uses

DashBoardViewModel ..> IRenterRepository : uses

DashBoardViewModel ..> IRackRepository : uses

DashBoardViewModel ..> IRentalContractRepository : uses

DashBoardViewModel ..> ISaleLineRepository : uses

EndRentContractViewModel ..> IRentalContractRepository : uses

EndRentContractViewModel ..> IRenterRepository : uses

RackViewModel ..> IRackRepository : uses

RackViewModel ..> IRentalContractRepository : uses

RackViewModel ..> IRenterRepository : uses

RackViewModel ..> IRepository~PaymentMethod~ : uses

RenterViewModel ..> IRenterRepository : uses

RenterViewModel ..> IRentalContractRepository : uses

RenterViewModel ..> ISaleLineRepository : uses

RenterViewModel ..> IRepository~PaymentMethod~ : uses

MonthlyStatementViewModel ..> IRenterRepository : uses

MonthlyStatementViewModel ..> IRentalContractRepository : uses

MonthlyStatementViewModel ..> ISaleLineRepository : uses